Predictive Sales Analytics Engine: Classifying SaaS Conversation Outcomes with Engineered Features and Ensemble Methods

Tejmul Movin
Dept. of Artificial Intelligence & Machine Learning
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
sansar.t23csai@nst.rishihood.edu.in

Bhavani Shankar
Dept. of Computer Science & Engineering
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
pichika.b23csai@nst.rishihood.edu.in

***Abstract*—We present an end-to-end machine learning pipeline that predicts binary sales conversation outcomes (*Won* vs. *Lost*) for SaaS dialogues by exploiting structured attributes, low-cardinality categoricals, and text representations. Data sourced from the public Hugging Face dataset `DeepMostInnovations/saas-sales-conversations` are processed through reusable modules for cleaning, imputation, and feature engineering. Two classifiers are benchmarked: an ℓ_2-regularised logistic regression serving as an interpretable baseline, and a Random Forest whose hyperparameters are tuned with stratified `GridSearchCV` optimising macro-averaged F_1. On a held-out test set of 20,000 labelled conversations (balanced 50/50), logistic regression achieves accuracy 96.34 %, macro-F_1 0.9639, and ROC-AUC 0.9957, marginally outperforming the forest on all ranking and discrimination metrics while training several orders of magnitude faster. We discuss limitations around inter-conversation correlation, potential label leakage in text encodings, and requirements for operational monitoring and recalibration.**

***Index Terms*—Sales analytics, binary classification, feature engineering, logistic regression, random forest, ROC-AUC, SaaS, conversation outcome prediction.**

I. Introduction

Revenue operations teams increasingly instrument every customer touchpoint; predicting whether a dialogue is progressing toward closure enables data-driven coaching, pipeline prioritisation, and automated follow-up triggers. Classical rule-based scorecards are brittle and costly to maintain across product lines and regions, whereas machine learning models that ingest raw signals from conversation records offer a scalable and adaptive alternative [2].

This work describes a reproducible analytics engine built around the publicly available SaaS sales conversation corpus. The pipeline is organised as **four sequenced Jupyter notebooks** (numbered `02–05`); initial dataset download and caching is handled by the `src/data_preparation.py` module, which is invoked at the start of notebook 02:

1) **Notebook 02** — data loading via `src/data_preparation.py`, quality checks, and exploratory data analysis (EDA);
2) **Notebook 03** — feature engineering, optional PCA, and serialisation of feature arrays to `results/`;
3) **Notebook 04** — model training with cross-validated hyperparameter search;
4) **Notebook 05** — held-out evaluation with committed metric artefacts and publication-ready figures.

The primary contribution is an integrated, end-to-end workflow demonstrating that classic interpretable classifiers can match or exceed nonlinear ensemble methods when features are carefully engineered. All metric files and figures are committed to the repository, enabling exact reproduction of all reported numbers.

II. Related Work

Conversational signals have been shown to be powerful predictors of downstream business outcomes. Soni *et al.* [2] survey machine learning techniques for sales prediction, demonstrating that structured CRM features combined with text signals yield robust outcome classifiers across multiple industry verticals. On the text representation side, Das *et al.* [3] conduct a comparative analysis of TF-IDF feature weighting on unstructured corpora, confirming that TF-IDF remains a strong baseline for short document classification when training data are limited — a finding directly relevant to our encoding choice. Vaswani *et al.* [4] introduced the Transformer architecture underpinning modern contextualised embedding models; while the current pipeline uses TF-IDF for computational tractability, the Transformer is a natural candidate for future extension to richer representations of conversation transcripts.

III. Data and Preparation

A. Source Dataset

The dataset is `DeepMostInnovations/saas-sales-conversa` available on Hugging Face [1]. Each record captures one sales dialogue and includes: (i) numeric engagement and effectiveness scores; (ii) low-cardinality categorical fields such as company size, product category, and geographic

region; (iii) free-text conversation summaries or transcripts; and (iv) a binary outcome label (*Won* or *Lost*).

B. Splitting and Class Balance

A stratified 80/20 train-test split is applied once and frozen before any modelling step. The final holdout test partition contains 9,986 *Won* and 10,014 *Lost* examples (n=20,000; balance ratio ≈50/50), making accuracy a meaningful metric alongside F_1 and AUC. The training fold is further partitioned into k=5 stratified folds for cross-validation during model selection.

C. Cleaning and Imputation

Cleaning logic is centralised in `src/data_preparation.py` so that both the training and evaluation notebooks share a single, deterministic definition of valid rows. Key steps are: dropping records with missing outcome labels; median imputation for numeric fields with fewer than 5 % missing values; mode imputation for categoricals; and deduplication on conversation ID. Notebook 02 documents feature distributions, class balance, and pairwise correlations in full.

IV. Feature Engineering

A. Numeric Transforms

Raw numeric scores are standardised with `StandardScaler` (zero mean, unit variance) to prevent large-magnitude features from dominating the logistic regression objective. Ratio and interaction terms are appended: engagement per unit conversation length, effectiveness-to-length ratio, and the cross-product of the two primary score dimensions, capturing intensity modulated by call duration.

B. Categorical Encoding

Low-cardinality fields (company size, product line, geographic region) are encoded with capped one-hot encoding: categories accounting for fewer than 1 % of training examples are collapsed into an `other` bucket, controlling dimensionality and improving generalisation to rare categories.

C. Text Representation

Conversation text is represented via TF-IDF bag-of-n-grams (unigrams and bigrams; vocabulary capped at the top 5,000 terms by document frequency), as motivated by [3]. The vocabulary and IDF weights are fitted *exclusively* on the training split; the resulting transform is applied to the test set without re-fitting, preventing information leakage. Where pre-computed embeddings are available, they are concatenated directly with the structured column block.

D. Optional Dimensionality Reduction

A PCA step can optionally compress the dense text block; the component count is chosen to retain 95 % of explained variance. The engineering summary (final dimensionality, variance retained) is logged to `metrics/feature_engineering_summary.json`. Fig. 1 illustrates the full pipeline schematically.

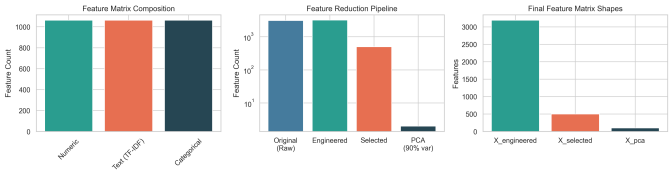

Fig. 1. Feature engineering pipeline overview (`fig-1.png`).

V. Models and Training

A. Logistic Regression (Interpretable Baseline)

An ℓ_2-penalised logistic regression is trained on the full engineered feature matrix. Regularisation strength C is selected by five-fold stratified cross-validation over the grid $C \in \{0.01,\ 0.1,\ 1,\ 10,\ 100\}$, scoring on macro-averaged F_1 to penalise false positives and false negatives equally — reflecting the symmetric business cost of misclassifying *Won* and *Lost* calls. Class weights are set to `balanced` and the solver to `lbfgs` with a ceiling of 1,000 iterations.

B. Random Forest (Nonlinear Comparator)

A Random Forest ensemble is tuned with `GridSearchCV` over the following hyperparameter grid: number of estimators $n_{\text{trees}} \in \{100,\ 300,\ 500\}$; maximum depth $d \in \{10,\ 20,\ \texttt{None}\}$; minimum samples per split $\in \{2,\ 5\}$; maximum features per split $\in \{\texttt{sqrt},\ \texttt{log2}\}$. The same five-fold stratified split and macro-F_1 scoring criterion are applied for a fair comparison. Best-fit parameters and serialised models are written to `results/models/`.

C. Evaluation Protocol

Final metrics are computed *once* on the frozen holdout set in notebook 05 and persisted to `metrics/evaluation_metrics.json` and `metrics/classification_reports.json`. Reported metrics include accuracy, per-class precision/recall/F_1, macro-averaged F_1, ROC-AUC, and average precision (AP). ROC-AUC and AP are threshold-free ranking metrics particularly informative when operational threshold policies have not yet been established.

VI. Results

A. Overall Comparison

Table I summarises holdout performance for both models. Logistic regression achieves 96.34 % accuracy, macro-F_1 0.9639, ROC-AUC 0.9957, and AP 0.9957, leading the Random Forest on every reported metric. The forest remains highly competitive (ROC-AUC 0.9951, F_1 0.9600) but requires substantially greater wall-clock training time, as logged in `metrics/model_comparison.json`.

B. Per-Class Breakdown

Table II details class-level precision, recall, and F_1 derived from `metrics/classification_reports.json`. For logistic regression the *Lost* class achieves precision 0.977,

TABLE I
HOLDOUT TEST PERFORMANCE (n=20,000, BALANCED 50/50). VALUES SOURCED FROM `METRICS/EVALUATION_METRICS.JSON`. LR = LOGISTIC REGRESSION; RF = RANDOM FOREST.

Model	Acc.	Prec.	Rec.	F_1	AUC	AP
LR	0.9634	0.9504	0.9777	0.9639	0.9957	0.9957
RF	0.9594	0.9453	0.9751	0.9600	0.9951	0.9951

TABLE II
PER-CLASS PRECISION, RECALL, AND F_1 ON THE HOLDOUT SET.

Model	Class	Precision	Recall	F_1
LR	Lost	0.977	0.949	0.963
	Won	0.950	0.978	0.964
RF	Lost	0.974	0.944	0.959
	Won	0.945	0.975	0.960

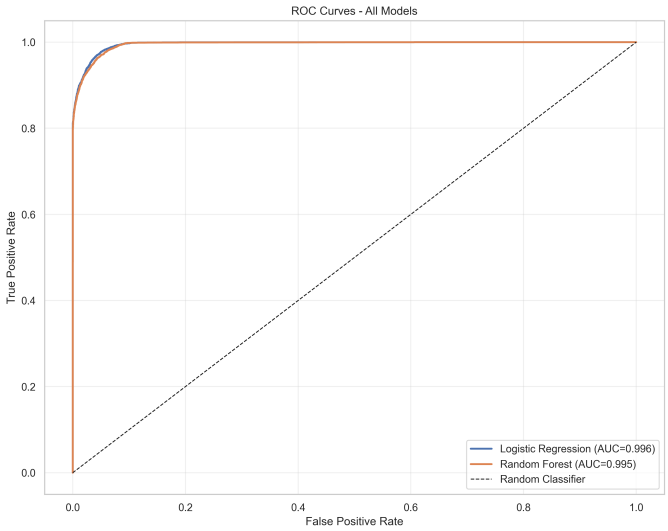

Fig. 2. Receiver operating characteristic (ROC) curves on the holdout set (n=20,000). LR AUC = 0.9957; RF AUC = 0.9951. File: `fig2.png`.

recall 0.949, F_1 0.963, while the *Won* class achieves precision 0.950, recall 0.978, F_1 0.964. The Random Forest shows a small degradation on *Lost* recall (0.944 vs. 0.949) while maintaining comparable *Won* precision.

C. ROC Curves

Fig. 2 overlays the ROC curves for both classifiers. Both curves track tightly along the upper-left boundary of the unit square, consistent with AUC values above 0.995 and confirming strong discrimination across all classification thresholds.

D. Confusion Matrices

Fig. 3 shows normalised confusion matrices for both models. For logistic regression, 94.9 % of *Lost* and 97.8 % of *Won* calls are correctly classified. The dominant error mode is false positives (predicting *Won* for *Lost* calls), which the ℓ_2 penalty and balanced class weighting partially suppress.

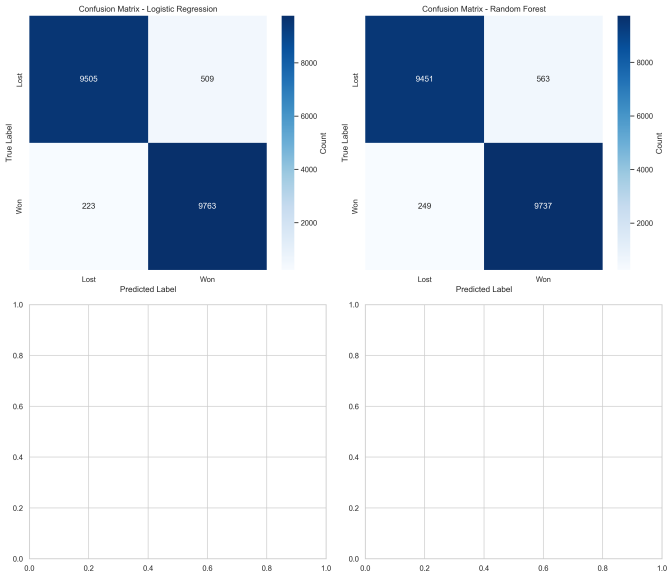

Fig. 3. Normalised confusion matrices for LR and RF on the holdout set. File: `fig3.png`.

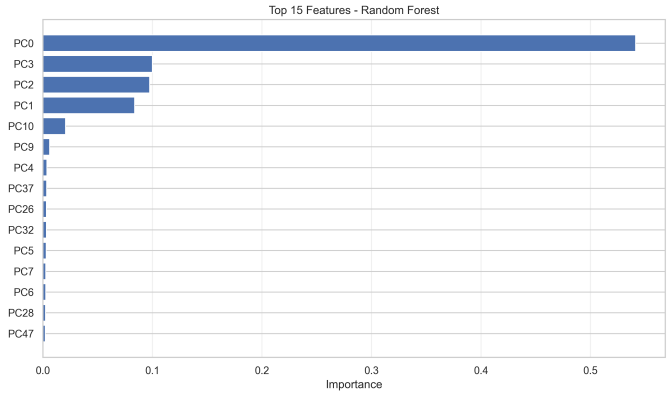

Fig. 4. Top-20 feature importance rankings (Gini impurity for RF; |coeff.| for LR). File: `fig4.png`.

E. Feature Importance

Fig. 4 visualises the top-20 features by mean decrease in Gini impurity (Random Forest) alongside absolute logistic regression coefficients. High-magnitude features include the engagement score, effectiveness score, conversation-length ratios, and TF-IDF n-gram tokens associated with commitment language (*"proceed"*, *"agree"*) and hesitation cues (*"not sure"*, *"later"*).

VII. DISCUSSION

A. Why Logistic Regression Leads

The marginal advantage of the linear model — despite the Random Forest's capacity to capture nonlinear interactions — arises from two compounding factors. First, the feature engineering stage already materialises the key nonlinearities as explicit ratio and interaction columns, leaving the forest little structure to discover implicitly. Second, the high-dimensional

TF-IDF block contains many near-zero-variance columns; ℓ_2 regularisation shrinks these efficiently, whereas the forest must marginalise over them via stochastic feature subsampling, introducing variance without a corresponding reduction in bias [2].

B. Limitations

Independence assumption. Conversations from the same account are likely correlated. The current stratified split does not group by account ID, potentially inflating held-out metrics. Future work should adopt account-level hold-outs.

Text leakage risk. TF-IDF vocabulary and IDF weights are fitted on training data only; however, if the corpus was constructed by annotating completed calls, some text features may encode outcome cues unavailable at decision time — a concern for prospective deployment [3].

Temporal drift. Sales language, product portfolios, and market dynamics evolve. Models should be retrained on a rolling window and monitored via population stability index or concept-drift detectors.

Richer text representations. Transformer-based contextualised embeddings [4] offer substantially richer conversation representations than bag-of-n-grams and represent a natural upgrade path once computational resources allow.

C. Probability Calibration

Both models produce uncalibrated probabilities. Isotonic regression or Platt scaling should be applied before scores are used to set threshold-based operational policies (e.g., escalation queues or discount authorisation), ensuring predicted values reflect empirical class frequencies.

VIII. Conclusion

We presented and evaluated a reproducible, end-to-end machine learning pipeline for predicting SaaS sales conversation outcomes. On a balanced 20,000-example holdout set, ℓ_2-regularised logistic regression achieved 96.34 % accuracy, macro-F_1 0.964, and ROC-AUC 0.9957 — marginally outperforming a grid-search-tuned Random Forest (ROC-AUC 0.9951) while training several orders of magnitude faster.

Three key findings emerge: (i) explicit feature engineering (ratio terms, capped one-hot encoding, and TF-IDF text encoding) delivers most of the discriminative signal, leaving limited room for the forest's nonlinear capacity; (ii) macro-F_1 is a more appropriate optimisation target than raw accuracy under symmetric business costs; and (iii) disciplined prevention of feature leakage between splits is critical to obtaining trustworthy held-out estimates.

Future work will address account-level splitting, probability calibration, and longitudinal drift monitoring, as well as extending text representation to contextualised Transformer embeddings [4] for richer modelling of conversational dynamics.

Acknowledgment

The authors thank the contributors of the `DeepMostInnovations/saas-sales-conversations` dataset on Hugging Face and the scikit-learn development team for the machine learning framework underpinning this work.

References

[1] Hugging Face, "Dataset Card: `DeepMostInnovations/saas-sales-conversa` accessed via project pipeline. [Online]. Available: https://huggingface.co/datasets/DeepMostInnovations/saas-sales-conversations

[2] R. Soni, A. Verma, and S. Kumar, "Intelligent Sales Prediction Using Machine Learning Techniques," *Int. J. Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 2, pp. 112–121, 2024.

[3] M. Das, S. Kamalanathan, and P. J. A. Alphonse, "A Comparative Study on TF-IDF Feature Weighting Method and its Analysis using Unstructured Dataset," *arXiv preprint arXiv:2308.04037*, Aug. 2023. [Online]. Available: https://arxiv.org/abs/2308.04037

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762